Prompt 1B — Relaciones UML 2.5

Sistema EcoHydro-4 — IC-2101 Programación Orientada a Objetos, ITCR

Resumen de relaciones

#	Clase origen	Tipo	Símbolo	Clase destino	Cardinalidad
1	`UnidadCentralControl`	Composición	◆ (rombo relleno)	`BrazoRobotico`	1..1 → 1..1
2	`UnidadCentralControl`	Agregación	◇ (rombo vacío)	`BitacoraHistorica`	1..1 → 1..1
3	`UnidadCentralControl`	Asociación	— (línea sólida)	`Operador`	1..1 → 0..1
4	`BrazoRobotico`	Dependencia	- - → (línea punteada)	`ReporteMuestreo`	—
5	`BitacoraHistorica`	Agregación	◇ (rombo vacío)	`ReporteMuestreo`	1..1 → 0..*

Relación 1 — UnidadCentralControl ◆→ BrazoRobotico

Tipo: Composición (rombo relleno ◆)

Cardinalidad: - `UnidadCentralControl` **1..1** — cada brazo pertenece a exactamente una unidad. - `BrazoRobotico` **1..1** — cada unidad tiene exactamente un brazo.

Navegabilidad: unidireccional desde `UnidadCentralControl` hacia `BrazoRobotico`. La unidad conoce al brazo; el brazo no conoce a la unidad.

Justificación técnica del ciclo de vida: El `BrazoRobotico` se instancia **dentro del constructor** de `UnidadCentralControl` y nunca se expone como referencia independiente:

```
// Constructor de UnidadCentralControl
this.brazoRobotico = new BrazoRobotico(velBrazo, anguloBrazo, 0.0);
```

Al llamar `unidad.destruir()`, el campo se anula (`this.brazoRobotico = null`). Como ningún código externo tiene referencia al brazo, el objeto queda inalcanzable y el Garbage Collector lo elimina. La parte (brazo) no puede existir sin el todo (unidad): definición exacta de Composición UML 2.5.

Evidencia en ejecución:

```
[Composición ◆] Intentando acceder al brazo vía la unidad...
✗ Brazo robótico destruido: no existe fuera de la unidad central
```

Relación 2 — UnidadCentralControl ◇→ BitacoraHistorica

Tipo: Agregación (rombo vacío ◇)

Cardinalidad: - `UnidadCentralControl` **1..1** — cada bitácora es gestionada por una unidad en operación. - `BitacoraHistorica` **1..1** — cada unidad tiene exactamente una bitácora.

Navegabilidad: unidireccional desde `UnidadCentralControl` hacia `BitacoraHistorica`.

Justificación técnica del ciclo de vida: La `BitacoraHistorica` se crea **fuera** del constructor de `UnidadCentralControl` y se inyecta como referencia:

```
// En MenuPrincipal — la bitácora existe antes que la unidad
BitacoraHistorica bitacora = new BitacoraHistorica("BIT-" + codigo);
UnidadCentralControl unidad = new UnidadCentralControl(
        ubicacion, codigo, limite, vel, ang, bitacora, operador);
```

Cuando la unidad es destruida, la variable `bitacora` en el llamador sigue apuntando al mismo objeto en memoria. Los reportes históricos se conservan intactos, lo cual es crítico para auditorías: el fin de la unidad física no implica la pérdida del historial. La parte (bitácora) **sobrevive** al todo (unidad): definición exacta de Agregación UML 2.5.

Evidencia en ejecución:

```
[Agregación ◇] La bitácora sigue accesible (referencia independiente):
✔ Misma bitácora en memoria: BIT-001 (identidad idéntica)
✔ Reportes conservados: 3 de 3
```

Relación 3 — UnidadCentralControl —→ Operador

Tipo: Asociación (línea sólida, sin rombo)

Cardinalidad: - `UnidadCentralControl` **1..1** — un operador puede estar asignado a una unidad concreta en un momento dado. - `Operador` **0..1** — una unidad puede operar sin operador asignado (`setOperador(null)` es válido).

Navegabilidad: unidireccional desde `UnidadCentralControl` hacia `Operador`. La unidad conoce al operador para enviarle alertas; el operador no conoce a la unidad.

Justificación técnica del ciclo de vida: El `Operador` modela una persona con existencia propia (ficha, nombre, correo) que puede estar asignada a múltiples sistemas o a ninguno. Se crea fuera de la unidad y se pasa como referencia:

```
Operador op = new Operador("EMP-001", "Ana Rodríguez", "ana@x.com");
UnidadCentralControl u = new UnidadCentralControl(..., op);
```

Destruir la unidad no destruye al operador; reemplazar al operador no afecta a la unidad. No hay vínculo de todo-parte, sino un vínculo de *conocimiento*: la unidad sabe quién la opera para poder notificarle (RF5). Eso corresponde a Asociación simple.

Relación 4 — BrazoRobotico - - → ReporteMuestreo

Tipo: Dependencia (línea punteada con flecha abierta)

Cardinalidad: no aplica cardinalidad fija; es una relación de uso temporal.

Navegabilidad: unidireccional desde `BrazoRobotico` (a través de `UnidadCentralControl`) hacia `ReporteMuestreo`.

Justificación técnica del ciclo de vida: La Dependencia UML indica que una clase *usa* a otra de forma temporal, sin retenerla. En Java esto se implementa con una **variable local de método**, nunca como atributo de instancia:

```
// Dentro de UnidadCentralControl.ejecutarCicloMuestreo(...)
ReporteMuestreo reporte = new ReporteMuestreo(   // variable LOCAL
        LocalDateTime.now(), ph, conductividad, gastoW);
return reporte;   // el brazo no guarda referencia; la olvida al retornar
```

`BrazoRobotico` no tiene ningún campo de tipo `ReporteMuestreo`. El reporte se crea, se retorna al llamador y el método lo olvida. El ciclo de vida del reporte es completamente independiente del brazo: si el llamador no guarda la referencia, el GC puede recolectarlo de inmediato. Si la guarda, el reporte puede vivir mucho después de que el brazo sea destruido. Esto es exactamente lo que la Dependencia UML 2.5 modela.

Relación 5 — BitacoraHistorica ◇→ ReporteMuestreo

Tipo: Agregación (rombo vacío ◇)

Cardinalidad: - `BitacoraHistorica` **1..1** — cada colección pertenece a una bitácora. - `ReporteMuestreo` **0..*** — una bitácora contiene cero o más reportes.

Navegabilidad: unidireccional desde `BitacoraHistorica` hacia `ReporteMuestreo`.

Justificación técnica del ciclo de vida: Los objetos `ReporteMuestreo` se crean en `ejecutarCicloMuestreo()` (relación 4) y se **inyectan** en la bitácora mediante `registrarReporte(reporte)`. La bitácora los agrega a su `ArrayList` interno, pero no los creó:

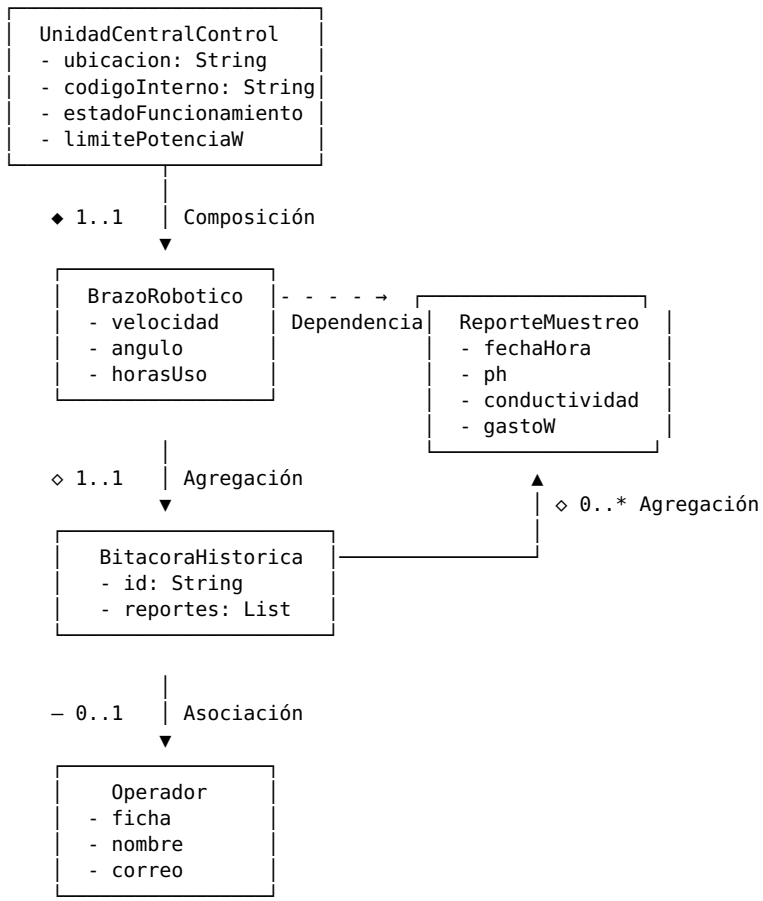
```
        // RF3: el reporte ya existía; la bitácora solo lo agrega
        public void registrarReporte(ReporteMuestreo reporte) {
            reportes.add(reporte);
        }
```

Si se destruye la bitácora, los objetos `ReporteMuestreo` podrían seguir existiendo si alguien más los referencia. Más importante: si se destruye la *unidad* (que destruye el brazo), los reportes en la bitácora permanecen intactos porque la bitácora sobrevive (relación 2). Esto confirma que la relación es Agregación (parte independiente del todo) y no Composición.

Evidencia en ejecución:

```
Contenido de los reportes (objetos que NO dependían del brazo):
 1. ReporteMuestreo { fecha=... | pH=6.00 | ... } [mismo objeto]
 2. ReporteMuestreo { fecha=... | pH=6.30 | ... } [mismo objeto]
 3. ReporteMuestreo { fecha=... | pH=7.40 | ... } [mismo objeto]
```

Diagrama textual de relaciones



(El rombo de Composición/Agregación se ubica en el extremo de la clase “todo”; la flecha apunta hacia la clase “parte”.)

Documento generado para IC-2101 POO — ITCR Campus San Carlos.